

Day 1 - Python Fundamentals

Personal revision book for the AI/ML + Semiconductor/EDA roadmap

Date: 13 September 2026 **Stage:** 1 - Python Programming

Today's outcome: you moved from writing a single `print()` statement to independently building small engineering programs with variables, calculations, decisions, and loops.

Resume point: Control flow is passed. Homework is pending review. Next major topic: functions.

Today	Mastery status
Program + syntax	Passed
Variables + data types	Passed
Operators + expressions	Passed
Input + type conversion	Passed
Boolean logic + decisions	Passed
Loops + control flow	Passed
Engineering applications	Built and tested

Contents

1. Program, Python, source code, syntax
2. print() and strings
3. Numeric types: int, float, str, bool
4. Variables, assignment, reassignment, expressions
5. Arithmetic operators
6. Input and type conversion
7. Errors: SyntaxError, TypeError, ValueError
8. Comparison operators and Boolean logic
9. if / elif / else and nested decisions
10. while loops
11. for loops, lists and indexing
12. range(), break, continue
13. Engineering programs built today
14. Mistakes and lessons
15. Mastery status + homework + resume point

1. Program, Python, source code, syntax

A **program** is a set of instructions for a computer. **Python** is a high-level programming language used to express those instructions. A **Python source file** normally ends in `.py`. **Syntax** means the rules that determine how Python code must be written.

When you run a file, the Python interpreter reads the source code and executes the instructions in it.

```
python hello.py
```

Command breakdown: `python` invokes the Python interpreter; `hello.py` identifies the source file to execute.

2. print() and strings

```
print("Hello")
```

`print` is a built-in Python function used to display a value. The parentheses contain the information passed to the function. `"Hello"` is a **string**, meaning text.

Quotation marks tell Python to treat the characters inside them as text. This difference matters: `25` is numeric, while `"25"` is text.

3. Numeric and Boolean data types

Example	Type	Meaning
15	int	integer / whole number
15.5	float	floating-point number
"15"	str	string / text
True / False	bool	Boolean truth value

A decimal point changes the type even when the numerical value looks equivalent:

```
15      # int
15.0    # float
```

4. Variables, assignment, reassignment, expressions

```
voltage = 12
resistance = 1000
current = voltage / resistance
```

`=` is the **assignment operator**. It evaluates the expression on the right and assigns the resulting value to the name on the left.

An **expression** is code that Python can evaluate to produce a value. In the line below, `voltage / resistance` is the expression.

```
power = voltage * current
```

Reassignment: a variable can later receive another value. Changing a variable does not automatically recalculate earlier assignments.

```
resistance = 1000
current = 12 / resistance
resistance = 2000
print(current) # 0.012
```

5. Arithmetic operators

Operator	Meaning	Example	Result
+	addition	5 + 3	8
-	subtraction	5 - 3	2
*	multiplication	5 * 3	15
/	true division	5 / 2	2.5
//	floor division	17 // 5	3
%	remainder / modulo	17 % 5	2

Engineering examples used today included Ohm's law and electrical power:

```
current = voltage / resistance
power = voltage * current
```

6. Input and type conversion

```
voltage = input("Enter the voltage value: ")
```

`input()` returns the user's response as a **string**. If the user types 24, Python initially has the text "24".

```
voltage = float(voltage)
resistance = int(resistance)
text = str(number)
```

The conversion functions you learned:

Function	Purpose	Example
<code>int()</code>	convert to integer	<code>int("25") -> 25</code>
<code>float()</code>	convert to float	<code>float("25.5") -> 25.5</code>
<code>str()</code>	convert to string	<code>str(25) -> "25"</code>

7. Error vocabulary

Error	Basic meaning	Example idea
<code>SyntaxError</code>	Python cannot parse the code structure	incomplete statement
<code>TypeError</code>	operation is invalid for the types involved	"15" / 3000
<code>ValueError</code>	type is acceptable but the specific value is invalid	<code>int("25.7")</code>

A key distinction from today: "15" / 3000 is syntactically valid Python, but the types do not support that division, so it is a `TypeError`.

8. Comparison operators and Boolean logic

Operator	Meaning
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
==	equal to
!=	not equal to

Critical distinction:

```
voltage = 12    # assignment  
voltage == 12  # comparison
```

Logic	Meaning
and	both conditions must be True
or	at least one condition must be True
not	reverses True and False

```
voltage <= 20 and voltage >= 10
```

9. if / elif / else and nested decisions

```
if voltage > 20:  
    print("Voltage too high")  
else:  
    print("Voltage is within safe limit")
```

Indentation matters. The indented statements belong to the block controlled by the condition.

```
if voltage > 20:  
    print("HIGH")  
elif voltage < 10:  
    print("LOW")  
else:  
    print("NORMAL")
```

An if / elif / else chain is checked from top to bottom. The first true branch runs and the remaining branches are skipped.

Nested decisions allow a second test only after the first condition passes:

```
if voltage <= 20:  
    if current <= 3:  
        print("Safe")
```

10. while loops

A while loop repeats while its condition remains true.

```
count = 1  
  
while count <= 3:  
    print(count)  
    count = count + 1
```

Step	count	Condition	Action
1	1	True	print 1, then count -> 2
2	2	True	print 2, then count -> 3
3	3	True	print 3, then count -> 4
4	4	False	loop ends

Key rule: a useful while loop typically needs a condition and an update that can eventually make the condition false.

11. for loops, lists and indexing

```
voltages = [8, 12, 17, 23, 19]

for voltage in voltages:
    print(voltage)
```

The loop variable `voltage` receives each list item one at a time.

```
voltages = [8, 12, 17, 23, 19]

print(voltages[0])
print(voltages[3])
```

Indexes start at **0**. For a five-item list, valid indexes are 0 through 4. Asking for index 5 causes an `IndexError`.

12. range(), break and continue

`range()` follows the general form `range(start, stop, step)`. The stop value is excluded.

Expression	Values
<code>range(5)</code>	0, 1, 2, 3, 4
<code>range(2, 5)</code>	2, 3, 4
<code>range(0, 11, 2)</code>	0, 2, 4, 6, 8, 10

break stops the loop completely.

```
for current in currents:
    if current > 3:
        print("FAULT")
        break
```

continue skips the current iteration and moves to the next item.

```
for current in currents:
    if current > 3:
        continue
    print(current)
```

13. Engineering programs built today

The point of today was not to practice syntax in isolation. Each concept was tied to a small engineering use case.

Project / exercise	What you practiced
Ohm's-law calculator	input, float, variables, arithmetic, print
Power + reference comparison	multiplication, subtraction, variables, output order
Voltage safety checker	if / else, comparisons, engineering limits
Three-range voltage classifier	if / elif / else
Device condition checker	and, nested conditions, multiple limits
Measurement scanner	lists, for loop, chained comparisons, break
Current monitor logic	for loop, ranges, conditions, break / continue

Why this matters for your career: the same control-flow ideas will later appear in waveform processing, datasheet validation, simulation checks, test automation, ML pipelines, and AI tool logic.

14. Mistakes made today - keep these in your revision notes

Mistake / confusion	Correct lesson
cd = create directory	cd means change directory; mkdir creates a directory
TypeError = SyntaxError	Code can be syntactically valid but invalid for the involved types
current auto-updates when resistance changes	Assignments store results at execution time; no automatic live dependency
input() gives a number	input() gives a string; convert before numeric operations
Last printed value = final loop variable value	The update can change the variable again before the condition fails
Using wrong data in a correct loop	Input data is part of correctness; inspect both algorithm and data

Also note: // was not tested until it had been explicitly taught. From now on, tests will cover what you have actually learned, and any new construct will be explained first.

15. Mastery status, homework, and tomorrow

Mastery gate: PASSED for variables, types, operators, input/conversion, Boolean logic, decisions, lists, loops, and control flow.

Homework - Engineering Current Monitor

```
currents = [0.4, 0.9, 1.8, 2.5, 3.2, 0.7, 4.1]
```

Rules:

- current < 1 -> LOW
- 1 <= current <= 3 -> NORMAL
- 3 < current < 4 -> HIGH
- current >= 4 -> FAULT and stop immediately

Tomorrow: first do a short recall test, review the homework, then start functions from first principles. We will increase the pace, but every new code element will still be explained.

Resume From Here: Stage 1 -> Functions. The GitHub repository is ECE_AI_ROADMAP.

Quick Revision Checklist

Before tomorrow, you should be able to answer these without looking at the notes:

1. What is Python source code and what does the Python interpreter do?
2. What is the difference between 15, 15.0, and "15"?
3. What does = do in Python?
4. What is an expression?
5. Why does input() require conversion for numeric calculations?
6. What is the difference between SyntaxError, TypeError, and ValueError?
7. What is the difference between = and ==?
8. When do and/or/not return True?
9. How does if/elif/else choose a branch?
10. How does a while loop stop?
11. How is a for loop different from a while loop?
12. Why do list indexes start at 0?
13. What does range(0, 11, 2) produce?
14. What is the difference between break and continue?